

DOCUMENTATION_WEEKOOK

Documentation Weekook V2 — Fonctionnalités & Règles de gestion

Table des matières

1. [Architecture générale](#)
 2. [Authentification & Sécurité](#)
 3. [Gestion des utilisateurs](#)
 4. [Parcours Kooker](#)
 5. [Services \(offres\)](#)
 6. [Recherche & Découverte](#)
 7. [Réservations \(Bookings\)](#)
 8. [Paiements & Stripe](#)
 9. [Avis \(Reviews\)](#)
 10. [Messagerie](#)
 11. [Favoris](#)
 12. [Disponibilités](#)
 13. [Témoignages](#)
 14. [Upload de fichiers](#)
 15. [Backoffice Admin](#)
 16. [Notifications email](#)
 17. [Tâches automatisées \(Cron\)](#)
 18. [Pages Frontend](#)
 19. [Environnements & Déploiement](#)
 20. [Conventions techniques](#)
-

1. Architecture générale

Composant	Technologie
Frontend	React 18 + TypeScript + Vite 6 + Tailwind CSS 4 + Shadcn/ui
Backend	Express.js + TypeScript
ORM	Prisma
Base de données	MySQL (distant)
Auth	JWT dans cookies httpOnly
Paielements	Stripe Connect Express
Emails	Resend API
Hébergement	VPS Hetzner (Ubuntu 24.04)
CI/CD	GitHub Actions
Process manager	PM2
Reverse proxy	Nginx + SSL Let's Encrypt

Monorepo avec npm workspaces : `client/` , `server/` , `shared/`

Base de données — 16 tables

`users` , `kooker_profiles` , `user_profiles` , `kooker_specialties` , `services` , `service_images` , `menu_items` , `bookings` , `payments` , `reviews` , `favorites` , `availabilities` , `messages` , `configs` , `testimonials`

2. Authentification & Sécurité

2.1 Inscription (POST /api/v1/auth/register)

Champ	Règle
<code>email</code>	Format email valide, unique en base (409 si doublon)
<code>password</code>	Minimum 6 caractères, hashé avec bcrypt 10 rounds
<code>firstName</code>	Requis, minimum 1 caractère
<code>lastName</code>	Requis, minimum 1 caractère

- Rate limiting : 50 requêtes / 15 min par IP
- Rôle initial : `user`
- Un JWT est généré et stocké dans un cookie `httpOnly` immédiatement

2.2 Connexion (POST `/api/v1/auth/login`)

- Rate limiting : 50 requêtes / 15 min par IP
- Message d'erreur unique pour email ou mot de passe incorrect (pas d'énumération)
- La réponse inclut `kookerProfileId` si l'utilisateur est kooker

2.3 Déconnexion (POST `/api/v1/auth/logout`)

- Supprime le cookie JWT
- Aucune authentification requise

2.4 Profil courant (GET `/api/v1/auth/me`)

- Retourne les infos de l'utilisateur connecté + `kookerProfileId`

2.5 Gestion des sessions (JWT + Cookies)

Paramètre	Valeur
Cookie <code>httpOnly</code>	<code>true</code>
Cookie <code>secure</code>	<code>true</code> en production, <code>false</code> en dev
Cookie <code>sameSite</code>	<code>strict</code>
Cookie <code>maxAge</code>	1 heure (timeout d'inactivité)
Durée max du JWT	7 jours
Sliding session	Le cookie est renouvelé à chaque requête authentifiée
Refresh token	Si le JWT a > 30 min, un nouveau token est signé

2.6 Cache d'authentification

- Cache en mémoire (TTL 60s) pour éviter une requête DB à chaque requête
- Stocke : `userId`, `email`, `role`, `kookerProfileId`
- Invalidé explicitement après changement de rôle ou de profil kooker

2.7 Middlewares de sécurité

Middleware	Rôle
<code>authenticate</code>	Vérifie le JWT, peuple <code>req.user</code> , gère le sliding session
<code>requireKooker</code>	Vérifie que <code>req.user.kookerProfileId</code> existe
<code>requireAdmin</code>	Vérifie que <code>req.user.role === 'admin'</code>
<code>validate(schema)</code>	Valide le body avec un schéma Zod, retourne 400 si invalide
<code>rateLimit(max, window)</code>	Limite par IP en mémoire

2.8 Headers de sécurité

Helmet.js avec 13 headers de sécurité HTTP. Body limité à 1 Mo (`express.json({ limit: '1mb' })`).

3. Gestion des utilisateurs

3.1 Modifier le profil (PUT /api/v1/users/profile)

Champ	Type	Règle
<code>firstName</code>	string	Optionnel
<code>lastName</code>	string	Optionnel
<code>phone</code>	string	Optionnel
<code>email</code>	string	Optionnel, format email valide, unicité vérifiée

- Si l'email change, un nouveau JWT est généré

3.2 Modifier l'avatar (PUT /api/v1/users/avatar)

- Champ `avatar` : URL (string)
- Pas de validation du format d'image

3.3 Profil d'accueil (UserProfile)

GET/PUT /api/v1/users/hosting-profile — Informations du domicile de l'utilisateur pour les kookers à domicile.

Section	Champs
Adresse	address , addressComplement , city , postalCode , country
Accès	accessCode , floor , intercom , parkingInfo
Cuisine	stoveType (gaz/induction/électrique/mixte), hasOven , hasDishwasher , tableCapacity (1-100), kitchenNotes
Alimentaire	dietaryRestrictions (array), allergies (array)
Notes	hostingNotes

- Upsert : crée le profil s'il n'existe pas, met à jour sinon
- Tous les champs sont optionnels

4. Parcours Kooker

4.1 Devenir Kooker (POST /api/v1/kookers/become)

Prérequis : Utilisateur authentifié, pas déjà kooker (409 sinon)

Champ	Type	Requis	Règle
bio	string	Non	Description publique
specialties	string[]	Oui	Au moins 1 spécialité
type	string[]	Oui	Types de service (KOOK, KOURS)
city	string	Oui	Minimum 1 caractère
experience	string	Non	Ex: "5 ans"
isCompany	boolean	Non	Défaut false

État initial du profil :

Champ	Valeur
<code>active</code>	<code>false</code> (en attente de validation)
<code>verified</code>	<code>false</code>
<code>featured</code>	<code>false</code>
<code>rating</code>	<code>0</code>
<code>reviewCount</code>	<code>0</code>
<code>stripeOnboardingComplete</code>	<code>false</code>

Actions déclenchées : 1. Création du `KookerProfile` 2. Rôle utilisateur mis à jour : `user` → `kooker`
3. Cache d'authentification invalidé 4. Redirection vers le dashboard kooker (pas de redirection Stripe)

4.2 Modifier le profil kooker (PUT /api/v1/kookers/profile)

Champs modifiables : `bio` , `specialties` , `type` , `city` , `experience` , `address` , `isCompany` — tous optionnels. Seul le kooker authentifié peut modifier son propre profil.

4.3 Validation du kooker

Règle : Un nouveau kooker est invisible (`active: false`) jusqu'à validation.

Validation manuelle (Admin)

L'admin peut activer/désactiver un kooker depuis le backoffice via les toggles `active` et `verified` .

Auto-validation automatique

Déclenchée quand le kooker termine son onboarding Stripe. Conditions (toutes requises) :

Condition	Détail
<code>active === false</code>	Pas déjà validé
<code>bio</code>	Non vide (après trim)
<code>city</code>	Non vide (après trim)
<code>specialties</code>	Array avec ≥ 1 élément
<code>type</code>	Array avec ≥ 1 élément
<code>stripeOnboardingComplete</code>	<code>true</code>

Si toutes les conditions sont remplies : - `active` → `true` - `verified` → `true` - Email envoyé à tous les admins : "Kooker auto-validé : {nom}"

Points de déclenchement : 1. Webhook Stripe `account.updated` (quand `charges_enabled` && `payouts_enabled`) 2. GET `/stripe/connect/status` (quand le kooker vérifie son statut sur le dashboard)

4.4 Visibilité

- **Recherche publique** : filtre `active: true` → les kookers inactifs sont invisibles
- **Profil public** (GET `/kookers/:id`) : accessible même si inactif (par URL directe)
- **Réservations** : les services d'un kooker inactif ne peuvent pas être réservés

5. Services (offres)

5.1 Créer un service (POST `/api/v1/services`)

Auth : Kooker authentifié

Champ	Type	Requis	Règle
<code>title</code>	string	Oui	Non vide
<code>type</code>	string[]	Oui	Au moins 1 type (KOOK, KOURS)
<code>priceInCents</code>	number	Oui	≥ 0 (stocké en centimes)
<code>durationMinutes</code>	number	Oui	≥ 1
<code>description</code>	string	Non	Max 2000 caractères
<code>minGuests</code>	number	Non	
<code>maxGuests</code>	number	Non	Défaut 1
<code>allergens</code>	string[]	Non	
<code>constraints</code>	string[]	Non	
<code>specialty</code>	string[]	Non	
<code>prepTimeMinutes</code>	number	Non	
<code>ingredientsIncluded</code>	boolean	Non	Défaut <code>false</code>
<code>equipmentProvided</code>	boolean	Non	Défaut <code>false</code>

Champ	Type	Requis	Règle
koursDifficulty	string	Non	Pour les KOURS
koursLocation	string	Non	Pour les KOURS
menuItems	object[]	Non	{ category, name, description? }
images	string[]	Non	URLs des images

- Service créé avec `active: true` par défaut
- Images et menuItems créés avec `sortOrder` correspondant à l'index dans le tableau

5.2 Modifier un service (PUT /api/v1/services/:id)

- Vérification de propriété : le service doit appartenir au kooker authentifié
- Tous les champs sont optionnels
- Si `images` ou `menuItems` fournis : les anciens sont **supprimés** et les nouveaux **recréés** (remplacement complet)

5.3 Supprimer un service (DELETE /api/v1/services/:id)

- Vérification de propriété
- Suppression en cascade : images et menuItems supprimés automatiquement

5.4 Activer/Désactiver

- Via PUT avec `{ active: boolean }`
- Un service inactif n'apparaît pas dans les résultats et ne peut pas être réservé

6. Recherche & Découverte

6.1 Recherche de prestations (GET /api/v1/services/search) ★ nouveau

L'interface de recherche est centrée sur les **prestations** (et non plus les kookers). Un kooker avec 3 offres = 3 cartes distinctes dans les résultats.

Filtres disponibles :

Paramètre	Type	Comportement
q	string	Recherche texte sur : titre service, description, nom kooker, spécialités

Paramètre	Type	Comportement
type	string	Contenu du champ JSON type du service (KOOK / COURS)
specialty	string	Spécialités du kookerProfile
city	string	kookerProfile.city (case-insensitive)
minPrice / maxPrice	number	Sur priceInCents du service (euros → centimes)
difficulty	string	koursDifficulty (COURS uniquement)
featured	boolean	kookerProfile.featured = true
page	number	Défaut 1
limit	number	Défaut 12

Logique de recherche :

1. Seuls les services active: true appartenant à un kooker active: true sont retournés
2. Tri : kookerProfile.featured DESC puis kookerProfile.rating DESC
3. Pagination SQL native
4. Include : images (isCardImage prioritaire), kookerProfile avec user (avatar, nom)

6.1b Recherche de kookers (GET /api/v1/kookers) — usage interne

Endpoint conservé pour les recherches internes (dashboard admin, etc.).

Filtres disponibles :

Paramètre	Type	Comportement
q	string	Recherche texte sur : nom complet, bio, ville, adresse, spécialités, types, titres/descriptions des services
type	string	Filtre exact sur le champ JSON type
specialty	string	Filtre exact sur le champ JSON specialties
city	string	Filtre par sous-chaîne (case-insensitive)
minPrice / maxPrice	number	Filtre sur le prix le plus bas des services actifs (en euros, converti en centimes)

Paramètre	Type	Comportement
<code>difficulty</code>	string	Filtre sur la difficulté des KOURS
<code>page</code>	number	Défaut 1
<code>limit</code>	number	Défaut 12, max 50

Logique de recherche :

1. Seuls les kookers `active: true` sont retournés
2. Tri : `featured DESC` puis `rating DESC`
3. Sans recherche texte (`q`) : filtrage en base de données + pagination SQL
4. Avec recherche texte : tous les kookers actifs chargés, filtrage JavaScript, pagination en mémoire
5. Les champs JSON (`specialties` , `type`) sont filtrés en JavaScript (pas de support SQL natif)

6.2 Profil kooker public (GET /api/v1/kookers/:id)

Données retournées : - Profil complet du kooker + infos utilisateur - Services actifs uniquement (avec images et menus triés par `sortOrder`) - Avis (`user_to_kooker` uniquement), triés par date DESC, avec infos du reviewer - Disponibilités futures (date ≥ aujourd'hui) - `confirmedSlots` : créneaux réservés (pending/confirmed/completed, date ≥ aujourd'hui) → `[{ date: YYYY-MM-DD, startTime: HH:MM, status }]`

7. Réservations (Bookings)

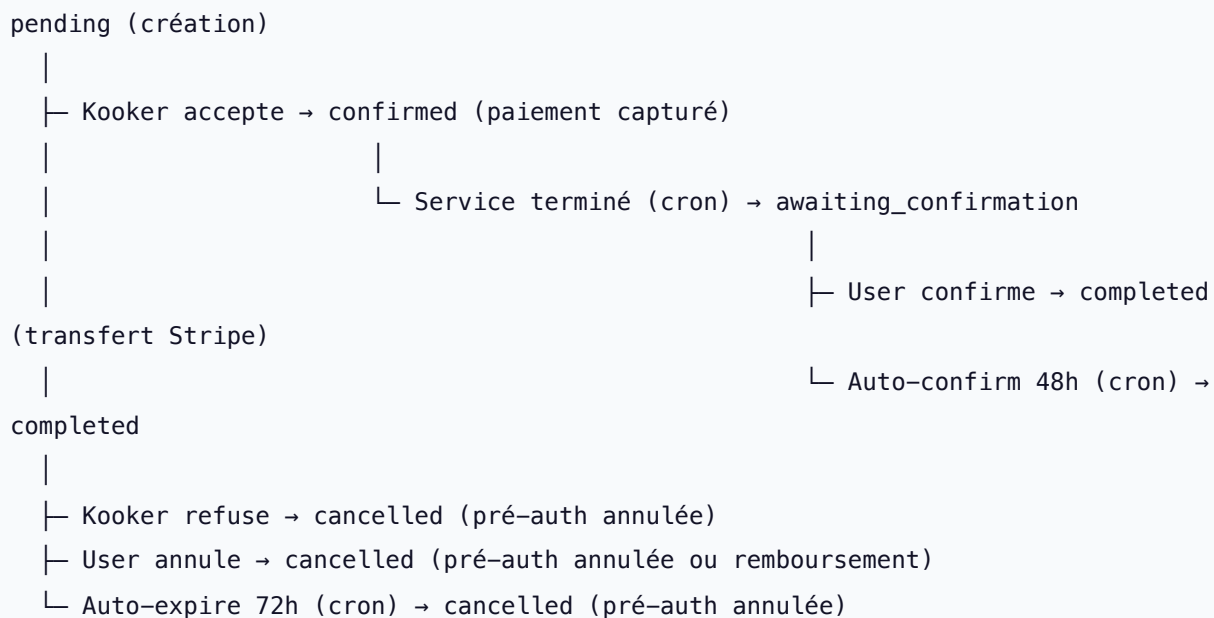
7.1 Créer une réservation (POST /api/v1/bookings)

Champ	Type	Requis	Règle
<code>serviceId</code>	number	Oui	Service doit exister et être actif
<code>date</code>	string	Oui	Ne peut pas être dans le passé
<code>startTime</code>	string	Oui	Format HH:MM
<code>guests</code>	number	Non	Défaut 1, ne peut pas dépasser <code>service.maxGuests</code>
<code>notes</code>	string	Non	

Règles de création : 1. Le service doit être `active: true` 2. Le kooker doit avoir terminé l'onboarding Stripe (`stripeOnboardingComplete: true` + `stripeAccountId`) 3. Calcul du prix : `service.priceInCents × guests` 4. Création d'un `PaymentIntent` Stripe en mode `capture_method: 'manual'` (pré-autorisation) 5. Statut initial : `pending` / `paymentStatus: pending_authorization` 6. Un enregistrement `Payment` (audit) est créé avec type `authorization` 7. Le `clientSecret` Stripe est retourné au frontend pour l'autorisation carte

Après confirmation du paiement (POST `/:id/confirm-payment`) : 1. Le `PaymentIntent` doit avoir le statut `requires_capture` (vérifié côté Stripe) 2. `paymentStatus` → `authorized` 3. Email envoyé au kooker : "Nouvelle demande de réservation" 4. Message système envoyé au kooker avec les détails

7.2 Cycle de vie des statuts



7.3 Réponse du kooker (PUT `/:id/status`)

Acceptation (`status: 'confirmed'`) : 1. Vérifie que la date n'est pas passée (sinon auto-annulation) 2. Capture le `PaymentIntent` Stripe (les fonds sont prélevés) 3. `paymentStatus` → `captured` 4. Email au user : "Votre réservation est confirmée" 5. Message système avec détails

Refus (`status: 'cancelled'`) : 1. Si `paymentStatus: authorized` → annulation du `PaymentIntent` (libère la pré-auth) 2. Si `paymentStatus: captured` → remboursement créé 3. Email au user : "Réservation annulée"

7.4 Modification d'une réservation (PUT `/:id`)

Par le user : - Uniquement si `status: 'pending'` - Champs modifiables : `date` , `startTime` , `guests` , `notes` - Si `guests` change : recalcul de `totalPriceInCents` - Email au kooker avec les changements

Par le kooker : - Interdit si `status: 'completed'` ou `'cancelled'` - Champs modifiables : `date` , `startTime` , `notes` (pas `guests`) - Email au user avec les changements

7.5 Annulation (PUT /:id/cancel)

- Le user OU le kooker peut annuler
- Interdit si déjà `cancelled` ou `completed`
- Si `paymentStatus: authorized` ou `pending_authorization` → annulation `PaymentIntent`
- Si `paymentStatus: captured` → remboursement client
- Email envoyé à l'autre partie

7.6 Confirmation de réalisation (PUT /:id/confirm-completion)

- Réservé au user (propriétaire de la réservation)
- Uniquement si `status: 'awaiting_confirmation'`
- Déclenche le transfert Stripe vers le kooker (voir section Paiements)
- `status` → `completed`

7.7 Accès aux réservations

Endpoint	Qui	Données
GET /bookings/my	User authentifié	Toutes ses réservations (en tant que client)
GET /bookings/kooker	Kooker authentifié	Toutes les réservations reçues
GET /bookings/:id	User ou kooker	Détail d'une réservation (403 si ni l'un ni l'autre)

8. Paiements & Stripe

8.1 Stripe Connect (comptes Express)

Paramètre	Valeur
Type de compte	Express
Pays	FR
MCC	5812 (Restaurants)

Paramètre	Valeur
Capacités	card_payments , transfers
business_type	individual (défaut) ou company (si isCompany: true)

8.2 Onboarding Stripe (POST /stripe/connect/onboard)

1. Si pas de compte Stripe → création d'un nouveau compte Express
2. Si compte existant mais business_type changé (toggle isCompany) → **nouveau** compte créé, ancien remplacé, stripeOnboardingComplete remis à false
3. Si compte existant et type identique → réutilisation
4. Retourne l'URL de l'interface Stripe pour compléter l'onboarding

8.3 Statut Stripe (GET /stripe/connect/status)

Retourne : { connected, chargesEnabled, payoutsEnabled, onboardingComplete }

Si l'onboarding vient de se terminer → met à jour stripeOnboardingComplete: true + déclenche l'auto-validation du kooker.

8.4 Cycle de vie du paiement

```

pending_authorization → authorized → captured → transferred
                        ↓           ↓
                    failed      refunded / cancelled

```

Statut	Signification
pending_authorization	PaymentIntent créé, en attente d'autorisation carte
authorized	Carte autorisée, fonds réservés (pas encore débités)
captured	Fonds débités du client (kooker a accepté)
transferred	Fonds transférés au kooker (moins commission)
cancelled	Pré-auth annulée (annulation avant capture)
refunded	Remboursement effectué (annulation après capture)
failed	Paiement échoué

8.5 Commission plateforme

- Taux par défaut : **20%** (configurable via le backoffice admin, table `configs`)
- Calcul : `commissionInCents = Math.round(totalPrice × rate / 100)`
- Montant transféré au kooker : `totalPrice - commissionInCents`
- Contraint entre 0% et 100%

8.6 Transfert Stripe

Déclenché par : 1. Confirmation de réalisation par le user 2. Auto-confirmation après 48h (cron) 3. Kooker qui passe manuellement en `completed`

Processus : 1. Vérifier `paymentStatus: 'captured'` 2. Calculer la commission 3. Transférer `totalPrice - commission` vers le `stripeAccountId` du kooker 4. `paymentStatus` → `transferred` 5. Enregistrement Payment audit avec type `transfer`

8.7 Webhooks Stripe (POST /stripe/webhook)

Événement	Action
<code>payment_intent.amount_capturable_updated</code>	<code>paymentStatus</code> → <code>authorized</code>
<code>payment_intent.payment_failed</code>	<code>paymentStatus</code> → <code>failed</code> , booking → <code>cancelled</code>
<code>account.updated</code>	Si <code>charges_enabled</code> && <code>payouts_enabled</code> → <code>stripeOnboardingComplete: true</code> + auto-validation

8.8 Audit trail (table `payments`)

Chaque action de paiement crée un enregistrement d'audit :

Champ	Valeur
<code>type</code>	<code>authorization</code> , <code>capture</code> , <code>transfer</code> , <code>refund</code> , <code>cancellation</code>
<code>status</code>	<code>pending</code> , <code>succeeded</code> , <code>failed</code>
<code>amountInCents</code>	Montant
<code>commissionInCents</code>	Commission (uniquement pour les transferts)
<code>metadata</code>	JSON avec détails (erreurs, taux, etc.)

9. Avis (Reviews)

9.1 Avis user → kooker (POST /api/v1/reviews)

Champ	Type	Requis	Règle
kookerProfileId	number	Oui	Le kooker doit exister
bookingId	number	Non	Si fourni : booking doit appartenir au user et être completed
rating	number	Oui	Entre 1 et 5
comment	string	Non	

Règles : - Impossible de s'auto-évaluer - Un seul avis par booking (409 si doublon) - Sans bookingId : un seul avis par user par kooker (legacy) - Après création : recalcul automatique de rating et reviewCount du kooker - $\text{rating} = \text{Math.round}(\text{moyenne} \times 10) / 10$ (1 décimale)

9.2 Avis kooker → user (POST /api/v1/reviews/kooker-to-user)

Champ	Type	Requis	Règle
bookingId	number	Oui	Booking doit être completed et appartenir au kooker
rating	number	Oui	Entre 1 et 5
comment	string	Non	

Prérequis : Le user doit avoir déjà laissé un avis user_to_kooker sur ce booking. L'avis kooker → user ne peut être donné qu'après.

9.3 Consultation des avis

Endpoint	Accès	Contenu
GET /reviews/kooker/:id	Public	Tous les avis user_to_kooker du kooker
GET /reviews/booking/:id	User ou kooker	Tous les avis (les deux types) d'un booking

10. Messagerie

10.1 Envoyer un message (POST /api/v1/messages)

Champ	Type	Requis	Règle
receiverId	number	Oui	Ne peut pas être soi-même
content	string	Oui	Minimum 1 caractère
kookerRecipientId	number	Non	Contexte kooker/booking

- Le destinataire doit exister (404 sinon)
- Email de notification envoyé au destinataire (cooldown 5 min par user)

10.2 Conversations (GET /api/v1/messages/conversations)

- Charge les 500 derniers messages (bidirectionnels)
- Groupe par interlocuteur
- Retourne par conversation : { user, lastMessage, unreadCount, kookerRecipientId }
- Trié par message le plus récent

10.3 Messages d'une conversation (GET /api/v1/messages/conversation/:userId)

- Retourne tous les messages entre les deux utilisateurs
- Marque automatiquement comme lus les messages reçus par l'utilisateur authentifié

10.4 Compteur non-lus (GET /api/v1/messages/unread-count)

- Retourne le nombre de messages non lus

10.5 Messages système

La plateforme envoie des messages automatiques dans la messagerie pour : - Nouvelle réservation (détails complets) - Réservation confirmée / refusée / annulée - Réservation modifiée (liste des changements) - Prestation terminée - Auto-confirimation après 48h - Expiration de réservation

11. Favoris

Endpoint	Action	Règle
GET /favorites	Lister	Retourne les kookers favoris avec profil et

Endpoint	Action	Règle
		services
POST /favorites/:kookerId	Ajouter	Le kooker doit exister. Idempotent (pas d'erreur si déjà favori)
DELETE /favorites/:kookerId	Retirer	404 si pas dans les favoris

- Contrainte d'unicité : (userId, kookerProfileId)

12. Disponibilités

GET /api/v1/availability/kooker/:id (Public)

- Retourne les disponibilités futures (date ≥ aujourd'hui)
- Trié par date ASC, puis startTime ASC

PUT /api/v1/availability (Kooker authentifié)

- **Mise à jour batch** : supprime toutes les disponibilités futures et recrée avec les nouvelles
- Format : { availabilities: [{ date, startTime, endTime, isAvailable? }] }
- isAvailable par défaut true

13. Témoignages

GET /api/v1/testimonials (Public)

- Retourne les témoignages avec featured: true
- Inclut le profil kooker et les infos utilisateur

POST /api/v1/testimonials (User authentifié)

Champ	Requis	Règle
authorName	Oui	Non vide
content	Oui	Minimum 10 caractères
rating	Oui	1-5

Champ	Requis	Règle
authorRole	Non	

- Créé avec `featured: false` (l'admin active manuellement)

14. Upload de fichiers

POST /api/v1/upload (User authentifié)

Paramètre	Valeur
Types acceptés	JPEG, PNG, WebP, GIF
Taille max	5 Mo
Stockage	Dossier <code>uploads/</code>
Nom de fichier	<code>{timestamp}-{random}.{extension}</code>

Retourne : `{ url: '/uploads/filename', filename, mimetype, size }`

15. Backoffice Admin

Accès : Routes sous `/api/v1/admin/*`, protégées par `authenticate` + `requireAdmin`.

15.1 Dashboard (GET /admin/stats)

Retourne : nombre d'utilisateurs, de kookers, de réservations, et le revenu total (somme des réservations confirmed/completed).

15.2 Gestion des utilisateurs

Endpoint	Action
GET /admin/users	Liste paginée avec recherche (email, nom), filtre par rôle
PUT /admin/users/:id	Modifier le rôle ou suspendre (<code>role: 'suspended'</code>)
DELETE /admin/users/:id	Supprimer un utilisateur (cascade)

15.3 Gestion des kookers

Endpoint	Action
GET /admin/kookers	Liste paginée avec recherche (ville, email, nom)
PUT /admin/kookers/:id	Toggle featured , verified , active

Flags : - featured : Coup de coeur, affiché en priorité dans les résultats - verified : Badge "vérifié" sur le profil - active : Visibilité dans la recherche publique

15.4 Gestion des avis

Endpoint	Action
GET /admin/reviews/kooker/:id	Voir les avis d'un kooker
DELETE /admin/reviews/:id	Supprimer un avis + recalcul automatique de la note moyenne

15.5 Gestion des réservations

Endpoint	Action
GET /admin/bookings	Liste paginée avec filtre par statut

15.6 Gestion des services

Endpoint	Action
GET /admin/services	Liste paginée de tous les services

15.7 Gestion des témoignages

Endpoint	Action
GET /admin/testimonials	Liste complète
PUT /admin/testimonials/:id	Modifier (featured , authorName , content , rating)
DELETE /admin/testimonials/:id	Supprimer

15.8 Configuration dynamique

Endpoint	Action
GET /admin/config	Toutes les configs (JSON parsé)
PUT /admin/config/:key	Upsert d'une config

Clés de configuration :

Clé	Défaut	Usage
<code>specialties</code>	Provençale, Méditerranéenne, Pâtisserie, Grillades, Végétarien, Fruits de mer, Italienne, Asiatique	Filtres de recherche
<code>cities</code>	Marseille, Aix-en-Provence, Cassis, La Ciotat, Toulon, Nice, Arles, Avignon	Filtres de recherche
<code>allergens</code>	14 allergènes réglementaires EU	Création de services
<code>serviceTypes</code>	KOOK, KOURS	Types de services
<code>platformCommission</code>	20 (%)	Commission sur les paiements

16. Notifications email

16.1 Système d'envoi

- **Provider** : Resend API
- **Expéditeur** : Weekook <notifications@weekook.com>
- **Cooldown** : 5 minutes par utilisateur (anti-spam)
- **Template** : HTML brandé avec couleur primaire `#c1a0fd`
- **Helpers** : `emailWrapper(icon, title, body, ctaUrl, ctaLabel)` + `infoBox(rows)`

16.2 Liste complète des emails (17 types)

Réservations

Email	Destinataire	Déclencheur
Nouvelle demande de réservation	Kooker	Paiement confirmé par le user
Réservation confirmée	User	Kooker accepte

Email	Destinataire	Déclencheur
Réservation annulée	User	Kooker refuse ou annule
Réservation annulée	Kooker	User annule
Réservation modifiée	Kooker	User modifie les détails
Réservation modifiée	User	Kooker modifie les détails

Rappels kooker (réservation en attente)

Email	Délai	Ton
Rappel 1	+4h	Notification simple
Rappel 2	+24h	"Répondez dans les 48h"
Rappel 3	+48h	Dernier avertissement, "24h restantes"

Expiration automatique

Email	Destinataire	Déclencheur
Réservation expirée	User	Auto-annulation après 72h ou date passée
Réservation expirée	Kooker	Idem

Confirmation post-prestation

Email	Délai	Message
Demande de confirmation	Service terminé	"Merci de confirmer la réalisation"
Rappel 1	+24h	Rappel doux
Rappel 2	+36h	Dernier rappel, "12h avant auto-confirmation"
Auto-confirmation	+48h	"Prestation confirmée automatiquement"

Paielement

Email	Destinataire	Déclencheur
Paielement en cours	Kooker	Prestation confirmée, transfert initié

Administration

Email	Destinataire	Déclencheur
Kooker auto-validé	Tous les admins	Auto-validation d'un kooker

Messagerie

Email	Destinataire	Déclencheur
Nouveau message	Destinataire du message	Réception d'un message (cooldown 5 min)

17. Tâches automatisées (Cron)

Toutes les tâches tournent en timezone `Europe/Paris` .

Tâche	Intervalle	Action
Transition confirmed → awaiting_confirmation	Toutes les 5 min	Vérifie si l'heure de fin de service est passée. Si oui : change le statut, envoie l'email de confirmation au user
Rappels de confirmation au user	Toutes les 30 min	Envoie les rappels 2 (+24h) et 3 (+36h) pour les bookings en <code>awaiting_confirmation</code>
Auto-confirmation	Toutes les 15 min	Si <code>awaiting_confirmation</code> depuis ≥ 48h : passe en <code>completed</code> , exécute le transfert Stripe, notifie les deux parties
Rappels au kooker (pending)	Toutes les 15 min	Rappels à +4h, +24h, +48h pour les réservations en

Tâche	Intervalle	Action
		attente avec paiement autorisé
Auto-expiration des pending	Toutes les 15 min	Si <code>pending</code> depuis $\geq 72h$ OU date passée : annule la pré-auth Stripe, passe en <code>cancelled</code> , notifie les deux parties

Timeline complète d'une réservation

T+0 Réservation créée (`pending` / `pending_authorization`)
 T+0 User confirme le paiement → `paymentStatus: authorized`
 → Email au kooker + message système

 T+4h [Cron] Rappel 1 au kooker
 T+24h [Cron] Rappel 2 au kooker ("48h pour répondre")
 T+48h [Cron] Rappel 3 au kooker ("dernier avertissement")
 T+72h [Cron] Auto-expiration → `cancelled`
 OU date passée → `cancelled`

ALTERNATIVE : Kooker accepte avant 72h
 → Capture du paiement, `status: confirmed`
 → Email au user

[Heure de fin de service]
 [Cron] Transition → `awaiting_confirmation`
 → Email au user : "Confirmez la réalisation"

+24h [Cron] Rappel 1 au user
 +36h [Cron] Rappel 2 au user (dernier)
 +48h [Cron] Auto-confirimation → `completed`
 → Transfert Stripe (montant - commission)
 → Email au kooker : "Paiement en cours"

ALTERNATIVE : User confirme manuellement avant 48h
 → Transfert Stripe immédiat
 → `completed`

APRÈS COMPLETION :

- User peut laisser un avis (user_to_kooker)
- Kooker peut répondre avec un avis (kooker_to_user) APRÈS l'avis du user

18. Pages Frontend

Pages publiques

Page	Route	Description
HomePage	/	Hero, barre de recherche, prestations vedettes (ServiceCard via GET /services/search?featured=true), témoignages, FAQ
SearchPage	/recherche	Recherche prestations avec filtres (type, spécialité, ville, prix, difficulté) — GET /services/search
ServiceDetailPage	/prestation/:id	Détail prestation : galerie, description, prix, ingrédients, encart kooker, CTA réserver
KookerProfilePage	/kooker/:id	Profil complet, services en accordéon, galerie d'images, avis, planning, réservation
LoginPage	/connexion	Split-screen login/register
BookingPage	/reservation	Formulaire de réservation — params ?service={id}&kooker={kookerProfileId}
BookingDetailPage	/reservation/:id	Détail réservation, actions contextuelles, modales d'avis/témoignage
PricingPage	/tarification	Tarification plateforme
AboutPage	/a-propos	À propos de Weekook
BenefitsPage	/avantages	Avantages pour les kookers/users
TrustPage	/confiance	Confiance et garantie
FaqPage	/faq	FAQ
NotFoundPage	*	Page 404

Pages utilisateur authentifié

Page	Route	Description
UserDashboardPage	/tableau-de-bord	Réservations (à venir/historique), favoris, carte profil
UserProfilePage	/profil	Édition du profil (nom, email, téléphone, adresse, cuisine)
MessagesPage	/messages	Conversations et messagerie
BecomeKookerPage	/devenir-kooker	Formulaire pour devenir kooker

Pages kooker

Page	Route	Description
KookerDashboardPage	/kooker-dashboard	4 onglets : Stats, Réservations, Services, Planning + bannière Stripe
CreateMenuPage	/creer-offre	Créer un service
EditMenuPage	/modifier-offre/:id	Modifier un service

Pages admin

Page	Route	Description
AdminDashboardPage	/admin	Stats globales (users, kookers, bookings, revenu)
AdminUsersPage	/admin/users	Gestion des utilisateurs
AdminKookersPage	/admin/kookers	Gestion des kookers (featured, verified, active)
AdminBookingsPage	/admin/bookings	Suivi des réservations
AdminServicesPage	/admin/services	Catalogue des services
AdminTestimonialsPage	/admin/testimonials	Gestion des témoignages
AdminConfigPage	/admin/config	Configuration dynamique (spécialités, villes, allergènes, commission)

19. Environnements & Déploiement

Architecture

Environnement	URL	Branche	Port	Base
Dev	dev.weekook.com	develop	3001	weekook_DEV
Validation	val.weekook.com	staging	3002	weekook_VAL
Production	weekook.com	main (futur)	3000	weekook_PROD

Serveur

- **VPS Hetzner** : 91.99.128.31
- **OS** : Ubuntu 24.04
- **Node** : v22.19.0
- **PM2** : Gestion des processus (weekook-dev , weekook-val , weekook-prod)
- **Nginx** : Reverse proxy + SSL Let's Encrypt

Démarrage serveur (app.ts)

Le serveur établit la connexion Prisma **avant** d'appeler `app.listen()` pour éliminer la latence de 10 s au premier appel.

```
prisma.$connect()      ← connexion pool établie (~5-10s, une seule fois)
  ↓
seedConfig()           ← upsert des valeurs de config par défaut
  ↓
app.listen(PORT)       ← serveur accepte les connexions (Prisma déjà prêt)
  ↓
process.send('ready')  ← signal PM2 → reload gracieux
  ↓
setInterval(30s)       ← keepalive SELECT 1 toutes les 30s
```

- Pool de connexions : `connection_limit=10` , `pool_timeout=5` (dans l'URL Prisma)
- Si `prisma.$connect()` échoue → `process.exit(1)` (PM2 redémarre le worker)

CI/CD (GitHub Actions)

Workflow	Branche	Dossier serveur
deploy-dev.yml	develop	/var/www/weekook-dev
deploy-val.yml	staging	/var/www/weekook-val

Processus de déploiement : 1. Push sur la branche → GitHub Actions déclenché 2. SSH vers le serveur 3. `git pull` → `npm install` → `prisma generate` → `prisma db push` → `npm run build` → `pm2 reload`

Workflow de promotion

```
develop → (auto-deploy) → dev.weekook.com
  ↓ merge into staging
staging → (auto-deploy) → val.weekook.com
  ↓ merge into main (futur)
main    → (auto-deploy) → weekook.com
```

20. Conventions techniques

Convention	Détail
Prix	Stockés en centimes en DB, divisés par 100 pour l’affichage
Durées	Stockées en minutes en DB
Champs JSON	<code>specialties</code> , <code>type</code> , <code>allergens</code> , <code>constraints</code> , <code>specialty</code> : arrays JSON dans MySQL
Réponses API	<code>{ success: true, data: ... }</code> ou <code>{ success: false, error: ... }</code>
User authentifié	<code>req.user.userId</code> après middleware <code>authenticate</code>
IDs affichés	Toujours formatés avec 5 chiffres : <code>#00042</code>
Design system	Primary <code>#c1a0fd</code> , BG <code>#f2f4fc</code> , text <code>#111125</code> , font Inter
Border radius	Cards <code>rounded-[20px]</code> , boutons/inputs <code>rounded-[12px]</code>
Padding responsive	<code>px-4 md:px-8 lg:px-[96px]</code>
Proxy Vite (dev)	<code>/api</code> et <code>/uploads</code> proxifiés vers <code>localhost:3001</code>

Convention	Détail
Module serveur	CommonJS (pas ESNext, pour compatibilité <code>__dirname</code>)